Домашняя работа № 3

RustFS, MapReduce и DuckDB: анализ вакансий PHP

Кулинич Павел Васильевич · Инженерия данных 1.3

Вариант 5: «Программист, разработчик на PHP». Расчёты выполнены 19.10.2026.

1. Данные и объектное хранилище

Показатель	Результат
Исходники	it_vacancies_2024.csv и hh.zip
CSV / JSON	229 388 / 10 000 записей
Вход / уникальные ID	239 388 / 229 388
PHP после фильтра	8 661 вакансий: 341 из JSON, 8 320 из CSV
RustFS	1.0.1; восемь папок-дисков; EC 4+4
Бакет и вход	hh-lake/variant5-php/input/all_sources.csv; 78 430 228 байт
Результаты	variant5-php/results/; частоты для L=5, 10, 30

Все 10 000 ID из hh.zip пересекаются с CSV. Сначала идут JSON, затем CSV; оставляется первая запись ID до фильтра PHP. Это правило согласования, а не доказательство большей точности JSON.

Фрагменты фактических журналов: RustFS и S3

```
2026-10-19 16:39:52 MSK
Docker client=29.8.0 server=29.8.0
rustfs 1.0.1
/data
/data1
/data2
/data3
/data4
/data5
/data6
/data7
/data8
HTTP/1.1 200 OK
{"status":"ok","service":"rustfs-
endpoint","timestamp":"2026-10-19T13:39:53.138781378Z[Etc/Unknown]","version":"1.0.1"}
PUT/GET SHA256 OK s3://hh-lake/variant5-php/input/all_sources.csv 78430228
6a7708659dcc8923bf834a13a34e6a350f02ebec7fdf0f2b6219d28651910280
PUT/GET SHA256 OK s3://hh-lake/variant5-php/input/php_vacancies.csv 2722797
5ac05e963dc5f99d2163837a0bb730ea6dec13f037f71d3e4cbabfcd22c74847
```

Фактический stdout RustFS и S3; загрузка проверена обратным GET и SHA-256. Панель набрана из журналов, не является снимком рабочего стола.

Полные команды: logs/00_environment.txt и logs/02_storage.txt. Восемь папок на одном Mac имеют общую точку отказа; это учебный стенд, не production-кластер.

2. Фильтр варианта и два способа расчёта

Включается роль «Программист, разработчик» и отдельный токен PHP/PHP5/PHP7/PHP8 с возможной минорной версией в названии либо такой тег целиком в навыках. Регистр не учитывается. Только PHPStorm, PHPUnit или упоминание в description не подходят.

Проверка отбора	Вакансий
PHP/версия в названии	5 381
PHP/версия только в навыках	3 280
Без навыков / максимум стека	1 002 / 30
Диапазон дат входа	2023-12-18 - 2024-09-14

Навыки: lower, trim, свёртка пробелов, distinct и лексикографическая сортировка. Синонимы не объединяются. L=5/10 - префиксы для эксперимента; L=30 - полный наблюдаемый стек для бизнес-анализа.

Собственный MapReduce

boto3 GetObject → StreamingBody → TextIOWrapper → csv.DictReader. UTF-8 и многострочные поля обрабатываются корректно. batches() удаляет дубли, фильтрует и выдаёт по 100 подходящих вакансий; mapper() перебирает сочетания размера 1, 2 и 3.

```
for chunk in batches(reader, 100):
local_counts = mapper(chunk, cap, True)
reduced.update(local_counts)
```

Baseline хранит все JSON-ключи до Reduce. Оптимизированный вариант объединяет tuple-ключи в Counter внутри чанка; сериализует только итоговые ключи. Это локальный однопроцессный MapReduce, не вычислительный кластер Hadoop/Spark.

Независимый SQL в DuckDB

DuckDB 1.5.6/httpfs читает тот же all_sources.csv из S3, не результаты MapReduce. row_number() выбирает первую запись ID; собственный SQL-предикат фильтрует PHP. unnest разворачивает навыки, GROUP BY считает частоты; self join при a.skill < b.skill и b.skill < c.skill формирует пары и тройки без перестановок. Полный запрос: sql/frequencies.sql.

Пограничные случаи

edge_cases.py: дубли ID и навыков, пробелы/регистр, пустой стек, перенос строки, другая роль, PHP8.2/PHP7, исключение PHPStorm/PHPUnit. Тестовые строки не входят в исследование. Данные архивные, преимущественно 2024 года, а не текущий рынок 2026 года.

3. Проверка частот и рост числа комбинаций

L	Навыки: сумма / уник.	Пары: сумма / уник.	Тройки: сумма / уник.
5	35 509 / 924	67 318 / 6 698	64 806 / 14 424
10	52 837 / 1 416	181 102 / 18 585	397 529 / 88 454
30	60 254 / 1 714	285 780 / 33 821	1 157 213 / 300 547

При m навыках: C(m,1)=m; C(m,2)=m(m-1)/2; C(m,3)=m(m-1)(m-2)/6. Для m=5: 5/10/10, для 10: 10/45/120, для 30: 30/435/4 060. Поэтому тройки растут быстрее пар. Суммы выше учитывают реальные длины стеков, а не максимальный m.

Все ключи и частоты baseline, combined и DuckDB совпали при каждом L. Суммы дополнительно проверены по формулам C(min(m,L),k): девять независимых равенств. В S3 проверены SHA-256 всех девяти словарей.

Самые частые навыки

Навык / комбинация	Вакансий	Доля
php	7 225	83,4%
mysql	3 756	43,4%
git	2 977	34,4%
javascript	2 779	32,1%
html	2 233	25,8%

Пары

Навык / комбинация	Вакансий	Доля
mysql + php	3 522	40,7%
git + php	2 819	32,5%
javascript + php	2 622	30,3%
html + php	2 128	24,6%
laravel + php	1 822	21,0%

Тройки

Навык / комбинация	Вакансий	Доля
git + mysql + php	1 646	19,0%
javascript + mysql + php	1 497	17,3%
html + javascript + php	1 392	16,1%
css + html + php	1 215	14,0%
html + mysql + php	1 195	13,8%

Частота - число вакансий с комбинацией; знаменатель доли - все 8 661 вакансии, включая пустые стеки. Полные результаты: results/verification.json и results/*_5/10/30.json.

4. Профилирование: все 27 запусков

Три размера стека × три подхода × три попытки в свежих процессах. Порядок подходов циклически меняется; DuckDB использует два потока.

perf_counter включает импорт движка, чтение S3, фильтр, нормализацию, агрегацию и итоговый словарь, в том числе проверку/загрузку httpfs. Запуск интерпретатора до таймера и запись итогового JSON не включены. ru_maxrss измеряет пик памяти клиента, включая нативную память DuckDB и запись JSON; RustFS/Docker не входят.

Все попытки: секунды / пиковые MiB

L	Подход	Попытка 1	Попытка 2	Попытка 3
5	baseline	1.1661 / 104.48	1.0934 / 101.84	1.1479 / 103.55
5	combined	1.0225 / 94.19	1.2204 / 91.84	1.0113 / 90.56
5	duckdb	0.5422 / 426.14	0.5018 / 402.03	0.5235 / 425.95
10	baseline	1.6582 / 168.09	1.6540 / 171.39	1.6562 / 165.98
10	combined	1.3191 / 136.56	1.3090 / 132.81	1.5187 / 148.41
10	duckdb	0.5879 / 430.78	0.5803 / 439.50	0.5869 / 436.50
30	baseline	2.8404 / 335.36	2.9442 / 338.34	2.8834 / 340.67
30	combined	1.9375 / 255.67	1.9913 / 261.89	2.0245 / 242.14
30	duckdb	0.9409 / 568.62	0.8938 / 579.66	0.8818 / 603.03

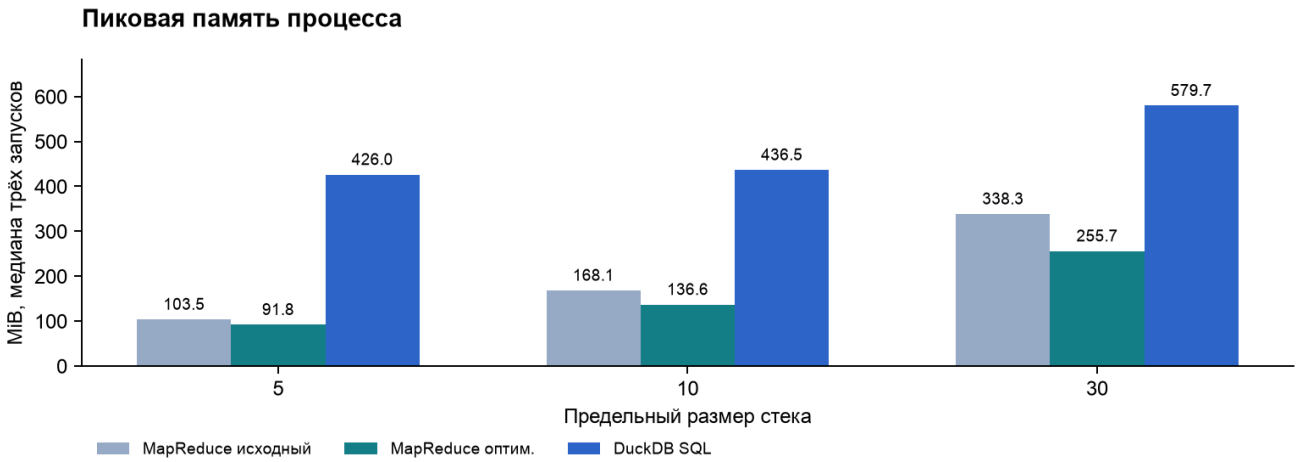
Медианы трёх попыток

L	Подход	Время, с	Пик RSS, MiB
5	baseline	1,1479	103,55
5	combined	1,0225	91,84
5	duckdb	0,5235	425,95
10	baseline	1,6562	168,09
10	combined	1,3191	136,56
10	duckdb	0,5869	436,50
30	baseline	2,8834	338,34
30	combined	1,9913	255,67
30	duckdb	0,8938	579,66

Полная точность: results/profiling_runs.csv; отдельные процессы: logs/profile_*.json. Min/max сохранены в profiling_summary.json. Никакие неудобные попытки не исключались; системный кэш не сбрасывался, это не cold-cache эксперимент.

5. Сравнение подходов и бонусная оптимизация





L	Время до / после, с	Снижение	RSS до / после, MiB	Снижение
5	1.148 / 1.023	10.9%	103.5 / 91.8	11.3%
10	1.656 / 1.319	20.4%	168.1 / 136.6	18.8%
30	2.883 / 1.991	30.9%	338.3 / 255.7	24.4%

На L=30 combiner снижает медианное время на 30,9%, пик RSS - на 24,4%. DuckDB быстрее baseline в 3,23 раза, но требует 579,7 MiB против 338,3 MiB.

Combiner сокращает повторы и сериализацию, но перечисление троек остаётся O(Σ C(m,3)). Итоговые ключи и seen по ID всё равно требуют памяти. Цифры относятся к этому Mac и этому объёму, не являются универсальной оценкой производительности.

6. Пары и тройки навыков: связь с зарплатой

Сопоставимые зарплаты: 4 202 из 8 661 (48,5%). Общая медиана - 127 750 руб. net.

Только RUR/RUB и gross=false. Зарплата - среднее двух положительных границ либо одна положительная граница; иначе NULL. Валюты и gross/net не смешиваются. Вакансии без зарплат остаются в частотах навыков.

Медиана «с» сравнивается с PHP-вакансиями без всей комбинации: Δ% = (median_with / median_without - 1) × 100%. Основной рейтинг n≥10; ниже top-3 каждой группы. Это наблюдаемая связь, а не причинное влияние навыка.

Пары: выше медианы контроля

Комбинация	n	Медиана с	Медиана без	Δ%
api + go	10	350 000	125 000	+180.0%
api + golang	10	350 000	125 000	+180.0%
backend + golang	11	350 000	125 000	+180.0%

Пары: ниже медианы контроля

Комбинация	n	Медиана с	Медиана без	Δ%
c++ + java	10	50 000	130 000	-61.5%
c++ + разработка по	10	50 000	130 000	-61.5%
crm + css	13	50 000	130 000	-61.5%

Тройки: выше медианы контроля

Комбинация	n	Медиана с	Медиана без	Δ%
api + go + php	10	350 000	125 000	+180.0%
api + go + sql	10	350 000	125 000	+180.0%
api + golang + php	10	350 000	125 000	+180.0%

Тройки: ниже медианы контроля

Комбинация	n	Медиана с	Медиана без	Δ%
cms wordpress + html + создание сайтов	12	45 000	130 000	-65.4%
1с-битрикс + css3 + figma	11	50 000	130 000	-61.5%
1с-битрикс + figma + javascript	13	50 000	130 000	-61.5%

Диапазон оценок: 75 - 1 500 000 руб. Ниже 10 000 руб. - 3 записи; без них медиана 129 000 руб. (+0,98%). Значения не исправлялись без оснований; инвертированных границ: 0.

Полный рейтинг, top-5, IQR, размер контроля и абсолютные разницы: results/salary_associations.csv и results/analysis.json.

7. Устойчивость результатов и независимые проверки

Порог n≥30 уменьшает влияние малых групп. В таблице - по два верхних и нижних сочетания каждого типа.

Тип	Комбинация	n	Медиана	Δ%
2, выше	php7 + rabbitmq	47	300 000	+140.0%
2, выше	backend + php7	41	262 500	+110.0%
2, ниже	c++ + php	36	70 000	-46.2%
2, ниже	cms wordpress + адаптивная верстка	45	70 000	-46.2%
3, выше	mysql + php7 + rabbitmq	42	325 000	+160.0%
3, выше	docker + php7 + rabbitmq	41	300 000	+140.0%
3, ниже	cms wordpress + javascript + администрирование сайтов	31	65 000	-50.0%
3, ниже	css3 + jquery + адаптивная верстка	32	65 000	-50.0%

Концентрация работодателей

api + go: 4 работодателей; крупнейший ID 2590553 даёт 6 из 10 зарплат (60,0%).

c++ + java: 3 работодателей; крупнейший ID 306478 даёт 7 из 10 зарплат (70,0%).

api + go + php: 4 работодателей; крупнейший ID 2590553 даёт 6 из 10 зарплат (60,0%).

cms wordpress + html + создание сайтов: 7 работодателей; крупнейший ID 1693767 даёт 5 из 12 зарплат (41,7%).

Высокие медианы backend/highload-сочетаний могут отражать опыт и состав работодателей; низкие медианы вёрстки/CMS не означают, что навык снижает доход. Связь также зависит от региона и выборочного раскрытия зарплат.

Аудит двух источников

Поле	Несовпадений среди общих ID
published_at	9 904
area	5
title	14
gross	1
skills_json	3

Приоритет JSON применяется до фильтра. Массовые различия дат не интерпретируются как динамика рынка; причины расхождений не предполагаются.

Контроль аналитики

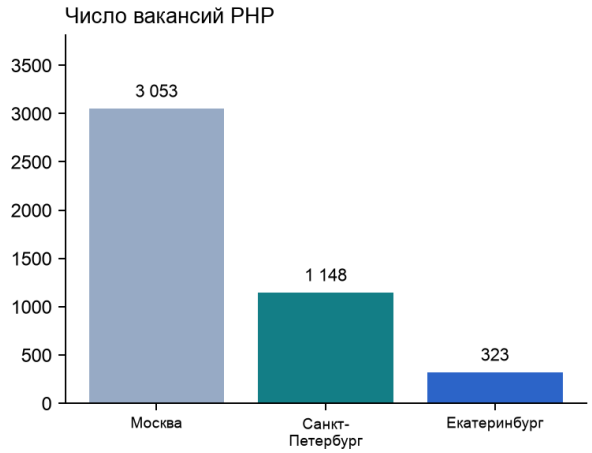

```
{
"all_passed": true,
"php_population": 8661,
"salary_n": 4202,
"salary_combinations_checked": 20,
"regions_checked": 3,
"regional_skill_rows_checked": 1821,
"identities_checked": 9
}
```

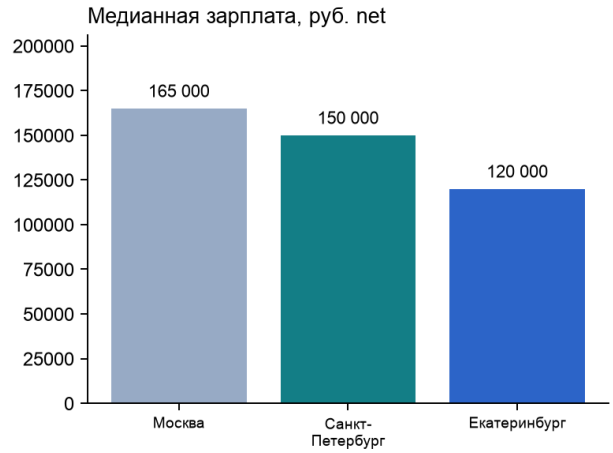

Python независимо проверил 20 выделенных зарплатных сочетаний, три региона, все 1 821 строки региональных навыков и девять комбинаторных равенств. Полные данные: results/quality.json и results/source_audit.json.

8. Сравнение трёх регионов

Москва, Санкт-Петербург и Екатеринбург выбраны по числу PHP-вакансий до анализа зарплат, а не по желаемому результату.

Регион	Все PHP	n зарплат	Раскрытие	Медиана	Q1 - Q3
Москва	3 053	1 306	42,8%	165 000	120 000 - 220 000
Санкт-Петербург	1 148	574	50,0%	150 000	100 000 - 193 750
Екатеринбург	323	209	64,7%	120 000	80 000 - 175 000





Спрос на навыки: число / доля вакансий региона

Навык	Москва	СПб	Екатеринбург
php	2444 / 80.0%	950 / 82.8%	270 / 83.6%
mysql	1247 / 40.9%	501 / 43.6%	158 / 48.9%
javascript	767 / 25.1%	342 / 29.8%	112 / 34.7%
git	988 / 32.4%	389 / 33.9%	94 / 29.1%
html	586 / 19.2%	247 / 21.5%	96 / 29.7%
css	412 / 13.5%	185 / 16.1%	72 / 22.3%
laravel	686 / 22.5%	301 / 26.2%	60 / 18.6%
docker	524 / 17.2%	192 / 16.7%	49 / 15.2%
1c-битрикс	365 / 12.0%	171 / 14.9%	38 / 11.8%

Знаменатель - все PHP-вакансии региона, включая пустые стеки. Полные таблицы: results/regions_all.csv, regional_skill_counts.csv и regional_skills_top10.csv.

Медиана Москвы - 165 000 руб.; Санкт-Петербург ниже на 9,1%, Екатеринбург - на 27,3%. Раскрытие зарплат различается, а Q1-Q3 пересекаются: сравнение не описывает доход всех работников.

Laravel чаще указан в СПб (26,2%), чем в Москве (22,5%) и Екатеринбурге (18,6%). Docker близок по долям: 17,2 / 16,7 / 15,2%; 1c-битрикс - 12,0 / 14,9 / 11,8%. Доли отделяют структуру навыков от размера рынка, но могут зависеть от состава работодателей.

9. Воспроизведение и итог

Каталог / файл	Содержание
src/ и sql/	Программы всех этапов и независимый SQL.
infra/compose.yaml	RustFS на восьми томах; секрет из окружения.
data/	Общий подготовленный вход и PHP-сегмент.
results/ и logs/	Частоты, аналитика, 27 профилей, аудиты и выводы команд.
report/ и figures/	PDF, графики и панели из журналов.
README.md / requirements.txt	Порядок запуска и зависимости.

Запуск из папки работы

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
export S3_ENDPOINT=http://127.0.0.1:9000
export S3_ACCESS_KEY=labadmin
export S3_SECRET_KEY='<локальный секрет>'
docker compose -f infra/compose.yaml up -d
.venv/bin/python src/run_lab.py --prepared
.venv/bin/python src/report.py
.venv/bin/python src/package.py
```

Вместо нового стенда можно запустить существующий rustfs-hw и передать --reuse-container rustfs-hw. Для оригиналов используются --csv и --zip. Секреты не печатаются и не сохраняются; Git и публикация не выполняются.

Контрольные суммы входных данных

it_vacancies_2024.csv
303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65

hh.zip
83e3b729d82da71b74b03ba3d35c27ff0490b885174467e574f05167dc7aa86c

all_sources.csv
6a7708659dcc8923bf834a13a34e6a350f02ebec7fdf0f2b6219d28651910280

Итог

Использованы оба источника: 239 388 строк, 229 388 уникальных ID, 8 661 PHP-вакансии. MapReduce и DuckDB совпали по всем частотам. На полном стеке: 1 714 навыков, 33 821 пар и 300 547 троек. Результаты трёх размеров стека сохранены в RustFS; проведены 27 замеров и независимые проверки.

Конец отчета